3D Gaussian Splatting for Real-Time Novel View Synthesis

STA306 Statistical Computing — End-Semester Project Report

Submitted by: Dwarkesh Nasit

AU2240155

Abstract

The report outlines the development and testing of 3D Gaussian Splatting (3DGS), a recent state-of-the-art technique in real-time novel view synthesis proposed by Kerbl et al. at SIGGRAPH 2023. The method is a three dimensional scene, and is a collection of anisotropic Gaussian primitives, parameterised by position, covariance, opacity, and spherical harmonic colour coefficients. These primitives are trained on a set of posed multi-view images with the help of differentiable rasterisation. The experiment was done on the Playroom indoor scene of the Deep Blending benchmark dataset, which consists of 225 calibrated training images with pre-computed Structure-from-Motion (SfM) camera parameters. The process of training was carried out 30,000 times on an NVIDIA T4 using Google Colab. The end result was a Peak Signal-to-Noise Ratio (PSNR) of 36.46 dB at convergence which is in line with published benchmarks of this scene. In this report, the theoretical background, experimental pipeline, quantitative results, qualitative observations, and lessons learned during the implementation process are described.

1. Introduction

A major issue in computer vision and computer graphics is the problem of novel view synthesis - to render photorealistic images of a scene when the camera views have been never photographed directly. It finds use in virtual reality, augmented reality, visual effects in cinema and architectural visualisation, cultural heritage preservation and immersive telepresence. The inherent challenge is that it is difficult to draw out a complete three dimensional representation of a scene based on a limited number of two dimensional measurements.

Classical photogrammetry and multi-view stereo techniques had been used in this area for decades, with explicit geometric forms of point clouds, meshes, and voxel grids. Although the techniques are interpretable and effective to render, they tend to have a hard time with view-dependent effects like specular highlights, translucency, and fine surface detail. Later on, implicit neural representations, most famously Neural Radiance Fields (NeRF) introduced by Mildenhall et al. in 2020, showed a radical improvement in the quality of reconstruction by representing the appearance and geometry of scenes in the weights of a multilayer perceptron. Nevertheless, NeRF rendering process needs hundreds of network evaluations per pixel, which in turn renders real-time performance on standard hardware impractical.

The bottleneck is dealt with by Kerbl, Kopanas, Leimkuhler and Drettakis in 3D Gaussian Splatting, which appears at SIGGRAPH 2023. Instead of asking a neural network to render at

render time, 3DGS models the scene directly as a set of three-dimensional Gaussian primitives, and renders them using a fast and differentiable tile-based rasteriser. The design achieves over 100 frames per second rendering at 1080p resolution with similar or better reconstruction quality than the NeRF based methods. The current paper recreates this pipeline on a held-out benchmark scene, analyzing the quantitative and qualitative aspects of the reconstruction.

2. Background and Related Work

2.1 Neural Radiance Fields (NeRF)

NeRF (Mildenhall et al., 2020) represents a scene as a continuous volumetric representation f(x, d) (c, sigma) where x is a spatial coordinate (3D) and d is a viewing direction, c is emitted RGB colour, and sigma is the density of the volume. This role is parameterised by a multilayer perceptron that is trained to reduce the photometric reconstruction loss over all the training views. The synthesis of a new view is done at inference by marching rays through the volume, numerically integrating colour contributions along each ray. The technique yields amazingly high-resolution geometry and view-dependent appearance reconstruction. Its main drawbacks are training time of up to hours and rendering time of several seconds per frame, both due to the dense neural network computations needed.

2.2 3D Gaussian Splatting

The 3DGS does not follow the implicit neural paradigm, but as a point-based scene representation is explicit. This scene is a set of N anisotropic 3D Gaussians, each defined by the following learnable parameters: a 3D mean position mu that defines the centre of the primitive; a 3D covariance matrix Sigma parameterised by a rotation matrix R and a scaling vector S as Sigma = R S S^T R T to ensure positive semi-definiteness; an opacity value alpha in [0,1]; and a

The rendering process is performed by projecting each 3D Gaussian onto the image plane with an affine approximation of the perspective projection, giving a 2D Gaussian splat. The splats are sorted by depth in screen-space tiles and the tiles are alpha-blended in order of front to back by a tile-based rasteriser. Since this rasteriser is a differentiable CUDA kernel, the gradients can be backpropagated through the rendering process and optimise the Gaussian parameters directly using photometric loss.

One important algorithmic element is adaptive density control: training entails periodic cloning of Gaussians in areas of high positional gradient (signifying under-reconstruction) and deforestation when their transparency becomes less than a particular threshold (signifying they are not contributing to the picture). This adaptive process is able to use the representation to add detail where it is the most applicable and drop redundant primitives.

3. Methodology

3.1 Dataset

This experiment was chosen based on the Playroom scene of the Deep Blending dataset (Hedman et al., 2018). Deep Blending dataset was specifically crafted to test new view synthesis techniques on a complex indoor setting, where there are also challenging lighting conditions, cluttered geometry, and view-dependent appearance. The Playroom scene comprises a domestic interior, where there is furniture, shelf, scattered objects and mixed artificial lighting.

The dataset contains 225 training images, which have a resolution of 1840 x 1216 pixels, with the pre-computed Structure-from-Motion (SfM) camera calibration data as COLMAP binary files (cameras.bin, images.bin, points3D.bin). The SfM reconstruction initialised 37,005 sparse 3D points, which were the initial positions of the Gaussian primitives at the start of the optimisation. The release of ready-computed SfM information meant that there was no need to actually run COLMAP and this is a computationally rigorous pre-processing step that normally takes one to three hours with this size of a dataset.

3.2 Implementation Environment

All the experiments were implemented on Google Collaboratory with an NVIDIA T4 and 15 GB VRAM, CUDA 12.8, and PyTorch 2.10. The core training algorithm was not modified and the official 3DGS implementation of the paper authors (graphdeco-inria/gaussian-splatting, commit main) was used. The environment needed patching of the simple-knn CUDA submodule to fix an error in compilation due to missing the float.h header in more recent CUDA toolchains - a compatibility issue added between CUDA 11.x and 12.x.

3.3 Training Configuration

The Adam optimiser and the learning rates given in the original paper were used to run the training with a total of 30,000 iterations. Images used as input were resized (--resolution 4) by four, before training, to save on the amount of memory used on the GPU and improve the speed of iteration without significantly compromising the quality of reconstruction at the desired output resolution. At iteration 7, 15,000 and 30,000 the checkpoints were saved to enable recovery in case of session interruption and to enable qualitative comparison of various stages of training.

The densification schedule had a gradient threshold of 0.0003, 200 iteration to densify and the densification process stopped at iteration 12,000 to enable the model to converge without additional structural alteration during the last phase of optimisation. The entire training execution took about 10 wall-clock minutes.

Table 1: Training Configuration Summary

Parameter	Value
Total iterations	30,000
Image resolution scale	4x downscale
Densification gradient threshold	0.0003
Densification interval	200 iterations
Densification stop iteration	12,000
Checkpoint intervals	7,000 / 15,000 / 30,000
GPU	NVIDIA T4 (15 GB VRAM)
Framework	PyTorch 2.10 + CUDA 12.8
Optimiser	Adam
Initial point cloud size	37,005 points
Training images	225
Total training time	~10 minutes

4. Results

4.1 Quantitative Evaluation

Peak Signal to Noise Ratio (PSNR) was used to assess the quality of a model, and this is a full-reference measure of image quality, which is defined as PSNR = 10 log10(maximum possible pixel range2 / mean squared error between rendered and ground-truth images). The larger the PSNR, the more the fidelity to the reference. Another metric, L1 photometric loss, was tracked during the training as the main optimisation goal.

The model showed steadily improved results in both measures at all checkpoints. With 7,000 iterations, an L1 error of 0.0228 and PSNR of 32.15 dB were achieved already, which is already in the range of high-quality reconstruction. By 15,000 iterations, the L1 loss had decreased to 0.0135. The model at the last checkpoint of 30,000 iterations had an L1 loss of 0.0097 and a PSNR of 36.46 dB, which is 4.31 dB better than the one at 7,000 iterations.

Table 2: Training Metrics Across Checkpoints

Iteration	L1 Loss	PSNR (dB)	Improvement vs. Iter 7k
7,000	0.02277	32.15	Baseline
15,000	0.01353	—	+~2.5 dB (est.)
30,000	0.00970	36.46	+4.31 dB

The PSNR of 36.46 dB is a good output of an indoor complex image. As a baseline, the initial 3DGS article with Deep Blending dataset results in PSNR values between 29 and 33 dB with full-resolution images and without downscaling. This larger value is in line with the resolution scale factor of 4 which downsizes the effective size of the image and hence the difficulty in the reconstruction task, but that the implementation is running as intended and the optimisation has reached a final value.

4.2 Qualitative Observations

The rendered outputs can be visually inspected to show that the model has managed to re-produce the key geometrical and photometrical characteristics of the Playroom scene. The spatial arrangement of the room is also correctly rendered, with furniture and wall surfaces, as well as relative depth of objects. Smooth surfaces with specular highlights and soft shadows are

represented with a reasonable fidelity showing that the spherical harmonic colour representation is effectively representing view-dependent appearance.

In some areas of fine geometric complexity (including the edges of thin objects and highly textured surfaces at oblique viewing angles), some amount of degradation is visible. This is in line with the established behaviour of representations based on Gaussian, which models surfaces as sweeps of soft volumetric splats, instead of hard geometric edges. Glass objects and other transparent and reflective surfaces are problematic to any representation of the scene using Lambertian or view-dependent Lambertian reflectance, and artefacts can be observed in these areas.

The rendered sequence exhibits temporal coherence between neighbouring viewpoints, and proves that the Gaussian representation has learned a geometrically consistent model of the scene, instead of being biased towards fitting specific training views.

5. Discussion

5.1 Comparison with NeRF

The main benefit of 3DGS compared to the NeRF-based methods, as it became in this experiment, is the saving of rendering time. The 225-frame sequence took around 14 seconds, or about 16 frames per second in a single forward pass - a relatively slow pace that would come close to real-time rendering even on the comparatively low-capacity T4 chip. It would take a few hours to render a similar sequence with a similar NeRF model. This distinction is essentially architectural in nature: 3DGS substitutes the per-ray network query with a parallelisable rasterisation operation that is inherently mapped to the hardware of a GPU.

Compared to state-of-the-art NeRF variants on restricted indoor scenes, 3DGS is typically comparable in terms of reconstruction quality, and much slower to train and infer with. The trade-off is the Gaussian representation has a larger memory footprint than implicit representations: a fully trained model can require a few hundred megabytes to a few gigabytes in memory depending on the complexity of the scenes it is trained on, and cannot generalise to new scenes without some re-training.

5.2 Practical Implementation Challenges

The process of implementation also had a few technical barriers, which should be recorded to make the process reproducible. The most enduring problem was that there was no compatibility between the CUDA sub modules included with the 3DGS codebase and the CUDA 12.8 runtime included in the Colab environment. In particular, the simple-knn extension would not compile as it was missing a header float.h, which was addressed by patching the source file beforehand. This type of problem, which occurs due to the fast development of CUDA toolchains, is a typical practical difficulty in the reproducibility of research in the field of deep learning using custom CUDA extensions.

The second type of challenge was related to the COLMAP feature extraction step, when trying to work with custom-captured images. The SIFT implementation in COLMAP is GPU-accelerated and thus needs an OpenGL context, which is not available in headless cloud platforms like Colab. CPU mode Running COLMAP in CPU mode is a plausible fallback although it comes with a large time cost - feature extraction of 166 images took about 41 minutes, and exhaustive feature matching was estimated to take a long time, more than two hours. This encouraged the use of the Deep Blending dataset that is pre-processed and does not require the COLMAP step.

These pragmatic aspects bring out a significant difference between the amount of algorithmic work in a research paper and the amount of engineering work needed to implement it in a limited computational medium. The 3DGS algorithm is a very pretty algorithm, and has a good theoretical motivation; converting it to an implemented version on free-tier cloud platform demanded some special focus on dependency management, compatibility with CUDA, time limitations on a session, and data format conventions.

6. Conclusion

The project was able to conduct the 3D Gaussian Splatting algorithm of novel view synthesis on the Playroom indoor scene successfully and tested it. The trained model reached a PSNR of 36.46 dB after 30,000 training steps, which is a good reconstruction quality in line with the results provided in the original article. The entire pipeline, including dataset preparation, training, and rendering, was run on a limited free-tier cloud computing setup, showing that this state-of-the-art technique can be made available to researchers without specialized access to a GPU infrastructure.

The 3DGS experiment validates the main argument of the 3DGS paper: that explicit Gaussian-based scene representations are able to be both high-quality visually and real-time-rendering, breaking a long-standing trade-off that constrained the practical value of neural rendering techniques. Differentiable tile-based rasterisation, with a combination of adaptive density control and the anisotropic Gaussian primitive, offers an inductive bias to the photometric reconstruction problem.

This study can be expanded in the future in a number of ways: it can be tested on outdoor unbounded scenes in which the MipNeRF360 parameterisation should be more effective; the relationship between the number of Gaussian primitives and reconstruction quality should be quantified; compression methods to reduce the storage footprint of trained model can be explored; the stability of the optimisation with different densification hyperparameters should be tested.

References

Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, *42*(4), 1–14. https://doi.org/10.1145/3592433

Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., & Ng, R. (2020). NeRF: Representing scenes as neural radiance fields for view synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)* (pp. 405–421). Springer. https://doi.org/10.1007/978-3-030-58452-8_24

Hedman, P., Philip, J., Price, T., Frahm, J.-M., Drettakis, G., & Brostow, G. (2018). Deep blending for free-viewpoint image-based rendering. *ACM Transactions on Graphics*, *37*(6), 1–15. https://doi.org/10.1145/3272127.3275084

Schönberger, J. L., & Frahm, J.-M. (2016). Structure-from-motion revisited. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 4104–4113). IEEE. https://doi.org/10.1109/CVPR.2016.445

Barron, J. T., Mildenhall, B., Verbin, D., Srinivasan, P. P., & Hedman, P. (2022). Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 5470–5479). IEEE. https://doi.org/10.1109/CVPR52688.2022.00539